

Development Plan — AI-Powered Robo-Advisor Platform

Course: Programming in Finance II (2026) — USI

Project: 2.1 AI-Powered Robo-Advisor Platform

Design version: v3.1 — Smart Single Portfolio (EU-Aware)

Plan version: 1.0

Period: Monday 27 April → Sunday 24 May 2026

Team size: 4 people

Introduction

This plan covers the four-week sprint from 27 April to 24 May 2026. It translates the v3.1 milestone structure into a concrete, day-by-day assignment for a four-person team. The original five milestones from the design document are compressed into four calendar weeks: the last two milestones (Frontend/Deploy and Hardening/Docs) are merged into a single, intensive final week.

Each week has one clear *milestone owner* and well-defined *hand-off points*, so no person is blocked waiting for someone else. Tasks are ordered to respect technical dependencies: data infrastructure is complete before the optimizer consumes it; the optimizer produces `OptimizationResult` objects before the LLM narrator consumes them; the full backend is stable before the frontend is wired up.

Team roles (fixed across all weeks):

Person	Primary role
P1	Backend / Data Engineering
P2	Quant / Portfolio Optimization
P3	ML / Risk Profiling
P4	Frontend / LLM / Documentation

Weekly Overview Table

Week	Dates	Milestone	Key deliverable to hand off
1	27 Apr - 3 May	Foundation	Repo live, DB schema, <code>universe_config.py</code> , <code>ValidatedDataLoader</code> with UCITS fallback, CI green

Week	Dates	Milestone	Key deliverable to hand off
2	4 May - 10 May	ML + Optimizer	Rule-based profiler (Phase A), HRP + Ledoit-Wolf on 8 hybrid ETFs, <code>OptimizationResult</code> with <code>risk_contributions</code>
3	11 May - 17 May	LLM Advisor + SCF	Validated chat with 4-step Validator + EU Awareness (Rule 9), backtest on 3 historical scenarios, SCF clustering Phase B if time allows
4	18 May - 24 May	Frontend + Deploy + Hardening + Docs	Streamlit app, cloud deploy, pytest suite, LaTeX PDF, AI agent PR, release v1.0

Week 1 — 27 April to 3 May

Milestone: Foundation

Goal: Every piece of shared infrastructure that the rest of the project depends on must be complete and reviewed by end of Sunday. No downstream work starts on unstable foundations.

Expected concrete outputs

Output	Owner	Required by
GitHub repository with agreed folder structure	P1	Everyone, immediately
<code>pyproject.toml</code> with pinned dependencies	P1	P2, P3, P4 from Week 2
CI pipeline (lint + pytest skeleton) passing	P1	All PRs from Week 2 onward
DB schema v3.1 (SQLite, all v3.1 fields including <code>ucits_fallback_tickers</code> , <code>etf_universe_type</code> , <code>regulatory_context</code>)	P1	P2 (audit trail inserts), P4 (LLM reads DB)
<code>universe_config.py</code> — 8 hybrid ETFs with UCITS primary/fallback map	P2	P1 (loader), P3 (optimizer)
<code>ValidatedDataLoader</code> with SHA-256 hash + UCITS fallback logic + <code>DataQualityReport</code>	P1	P2 from Week 2
<code>AGENTS.md</code> draft (agent roles described, PR strategy outlined)	P4	Criterion 5 — must be visible from day 1

Output	Owner	Required by
<code>README.md</code> skeleton (installation, usage placeholder, API placeholder)	P4	Prof's minimum documentation requirement
Architecture Decision Records folder (<code>docs/adr/</code>) with ADR-001 (why HRP over pure MV)	P4	Academic PDF, Week 4

Task breakdown by person

P1 — Backend / Data Engineering

- Mon-Tue:** Create GitHub repo; agree folder structure with team; push `pyproject.toml` with core dependencies (`yfinance`, `pandas`, `scipy`, `PyPortfolioOpt`, `fastapi`, `streamlit`, `anthropic`, `pytest`, `ruff`). Set up branch protection (require PR + 1 review before merge to `main`).
- Wed-Thu:** Implement `ValidatedDataLoader` (`backend/data/loader.py`):
 - `yf.download` with `auto_adjust=True`
 - NaN gate (`NAN_THRESHOLD = 0.02`), `ffill(limit=2)`, `dropna()`
 - UTC conversion
 - SHA-256 hash of serialised prices
 - UCITS fallback logic: if primary ticker fails yfinance fetch, swap to fallback from `universe_config.py`, record in `fallback_tickers_applied`
 - Return `(pd.DataFrame, DataQualityReport)`
- Fri-Sat:** Write DB schema v3.1 (`backend/data/snapshots.py`): `recommendations` table with all v3.1 fields. Write `test_data_loader.py` with at least 2 happy-path tests. Configure CI (`/.github/workflows/ci.yml`): `ruff check` + `pytest` on push.
- Sun:** Code review of P2's `universe_config.py`; confirm loader works end-to-end with real tickers.

Blocker for others: `ValidatedDataLoader` and DB schema must be merged by Friday so P2 can start optimizer integration on Monday of Week 2.

P2 — Quant / Portfolio Optimization

- Mon-Tue:** Define the 8 hybrid ETF universe (3 UCITS + 5 US equivalents). Write `universe_config.py` (`backend/data/universe_config.py`) with:

- `PRIMARY_TICKERS` list
 - `FALLBACK_MAP: dict[str, str]` (UCITS → US equivalent)
 - Helper functions: `get_primary_tickers()`, `get_fallback_map()`, `get_ucits_tickers()`
 - Inline comments justifying each ticker choice (geographic coverage, liquidity, expense ratio).
 - **Wed-Thu:** Draft the `OptimizationResult` TypedDict/dataclass (`backend/optimizer/hrp.py`) with all fields: `weights`, `risk_contributions`, `expected_return`, `volatility`, `sharpe`, `method` ("HRP" or "MV"). This interface contract must be final by Thursday so P3 can design inputs around it.
 - **Fri-Sun:** Begin Ledoit-Wolf shrinkage scaffold using `sklearn.covariance.LedgerWolf`. Validate that returns matrix computes correctly on downloaded data. Write `test_optimizer.py` stub with 1 test.
-

P3 — ML / Risk Profiling

- **Mon-Tue:** Write the questionnaire schema: define the 7-10 questions that capture investor characteristics (age, income bracket, investment horizon, loss tolerance, prior investment experience, liquidity needs). Document each question and its mapping rationale. This is the input to the Phase A rule-based profiler.
 - **Wed-Thu:** Write `backend/ml/profiler/rule_based.py` — Grable-Lytton inspired rule-based classifier that maps questionnaire answers to one of three profile labels (`CONSERVATIVE`, `MODERATE`, `AGGRESSIVE`). Include confidence score (deterministic: 1.0 for clear cases, 0.7 for borderline). This is the Phase A fallback that must work even if GBM is not ready.
 - **Fri-Sun:** Download SCF 2022 Summary Extract (CSV). Inspect columns, identify relevant behavioural variables (`NETWORTH`, `INCOME`, `SAVRES`, `RISKTOL`, etc.). Write `backend/ml/profiler/scf_pipeline.py` scaffold with data loading and weight column identification. Document choices in `docs/adr/ADR-002-scf-preprocessing.md`.
-

P4 — Frontend / LLM / Documentation

- **Mon-Tue:** Write `AGENTS.md`: describe at least three agent roles (Documentation Agent, Test Generation Agent, Code Review Agent). Describe the planned AI agent PR strategy (GitHub Actions + Claude API generating docstrings for `optimizer/` module). This document is criterion 5 evidence — treat it as a first-class deliverable.

- **Wed-Thu:** Write `README.md` skeleton with all required sections (project overview, installation with `uv`, usage examples placeholder, API documentation placeholder, link to GitHub). Also create the `docs/` folder structure and write `docs/architecture.md` with a high-level diagram description.
- **Fri-Sun:** Research and prototype the Ground Truth JSON schema that the LLM narrator will consume (Week 3). Draft the `regulatory_context` block structure. Write `docs/adr/ADR-001-hrp-over-mv.md`. Begin LaTeX document skeleton (title, abstract placeholder, section headings).

Dependencies this week

P2 (`universe_config.py`) → P1 (`ValidatedDataLoader` uses fallback map)
P3 (questionnaire schema) → P2 (optimizer needs profile label to apply tilt)
P4 (`AGENTS.md`) → independent, can proceed in parallel

Hand-off by end of Sunday 3 May:

- ☐ `main` branch: repo structure, CI green, `ValidatedDataLoader` merged, DB schema merged
- ☐ `universe_config.py` merged (P2 → P1)
- ☐ Rule-based profiler scaffold committed (P3)
- ☐ `AGENTS.md` and `README.md` skeleton committed (P4)

Week 2 — 4 May to 10 May

Milestone: ML + Optimizer

Goal: By end of Sunday, the core quantitative engine is functional: the rule-based profiler classifies a user, the HRP optimizer runs on real downloaded data with Ledoit-Wolf shrinkage, and the `OptimizationResult` object (including `risk_contributions`) is returned correctly. This is the backbone that Week 3 builds on.

Expected concrete outputs

Output	Owner	Required by
Rule-based profiler, Phase A — fully tested	P3	P2 (profile-to-tilt mapping), P4 (LLM context)
HRP implementation with Ledoit-Wolf on 8 hybrid ETFs	P2	P4 (LLM narrator), P1 (API endpoint)

Output	Owner	Required by
<code>OptimizationResult</code> with <code>risk_contributions</code>	P2	P4 (Ground Truth JSON), backend tests
MV tab scaffold (Markowitz benchmark)	P2	Week 4 frontend tabs
FastAPI endpoints: <code>/profile</code> , <code>/optimize</code>	P1	P4 (chat UI needs these), P3 (SCF pipeline calls <code>/profile</code>)
<code>test_profiler.py</code> and <code>test_optimizer.py</code> passing	P3, P2	CI stays green
SCF clustering Phase A labels committed	P3	Week 3 GBM training

Task breakdown by person

P1 — Backend / Data Engineering

- **Mon-Tue:** Wire `ValidatedDataLoader` into FastAPI. Implement `/optimize` endpoint: accepts `profile_label` + optional `tickers` override, calls optimizer, stores result in DB, returns `OptimizationResult` as JSON. Add Pydantic request/response models with full type hints.
- **Wed-Thu:** Implement `/profile` endpoint: accepts questionnaire JSON, calls rule-based profiler, returns `profile_label` + `confidence`. Add rate limiting via `slowapi` (this satisfies the "Creating your own API" standard criterion). Write integration test for both endpoints.
- **Fri-Sun:** Write `backend/data/snapshots.py` insert logic — every `/optimize` call writes a full row to `recommendations` (including `market_data_hash`, `ucits_tickers_used`, `fallback_tickers_applied`). Verify DB inserts end-to-end. Fix any CI failures.

P2 — Quant / Portfolio Optimization

- **Mon-Tue:** Complete `hrp.py`:
 - Download prices via `ValidatedDataLoader`
 - Compute log returns
 - Apply Ledoit-Wolf shrinkage to the sample covariance matrix
 - Run HRP (hierarchical clustering → quasi-diagonalisation → recursive bisection)
 - Apply profile-based tilt: AGGRESSIVE gets `+0.05` equity cluster, CONSERVATIVE gets `+0.05` bond/cash cluster

- Enforce box constraints `(0.03, 0.40)` per asset, `(0.10, 0.60)` per cluster
 - Return fully populated `OptimizationResult`
 - **Wed-Thu:** Implement `risk_metrics.py`: compute individual asset risk contributions from the final weight vector and the shrunk covariance matrix. These go into `OptimizationResult.risk_contributions`. Also compute ex-ante volatility, expected return (from sample means), and Sharpe ratio.
 - **Fri-Sun:** Implement `markowitz.py` as MV benchmark (max Sharpe, same constraints). This is the second tab in the frontend (Week 4). Write `test_optimizer.py` with at least 3 tests (HRP weights sum to 1, all weights within bounds, risk contributions sum to 1).
-

P3 — ML / Risk Profiling

- **Mon-Tue:** Complete `rule_based.py` — finalize scoring logic, edge case handling, unit tests in `test_profiler.py` (at least 3 test cases per profile label). The profiler must be importable and callable by P1's `/profile` endpoint by Tuesday.
 - **Wed-Thu:** Work on `scf_pipeline.py`:
 - Load SCF 2022 Summary Extract with sample weights (`wgt`)
 - Select behavioural feature columns (equity share, bond share, liquid assets ratio, net-worth-to-income ratio)
 - Apply sample weights to avoid high-net-worth individual bias
 - Use `implicate = 1` (document as a limitation)
 - Standardise features; prepare for clustering
 - **Fri-Sun:** Run K-Means / GMM clustering on SCF features (3 clusters → map to CONSERVATIVE / MODERATE / AGGRESSIVE). Validate cluster separation. Commit cluster labels and centroid description to `backend/ml/profiler/clustering.py`. Prepare training dataset for GBM (Week 3).
-

P4 — Frontend / LLM / Documentation

- **Mon-Tue:** Design the Ground Truth JSON schema completely. Every field that the LLM narrator will use must be defined: `weights`, `risk_contributions`, `profile_label`, `profile_confidence`, `top_drivers` (from rule-based for now), `regulatory_context` (with `profiler_us_centric_caveat`, `mifid_disclaimer`, `currency_risk_note`, `etf_ucits_eligible`, `hfcs_note`). Commit as `docs/ground_truth_schema.md`.
- **Wed-Thu:** Write the LLM system prompt first draft (`backend/llm/prompts/`): include all 9 rules from v3.1 (especially Rule 9 EU Awareness), forbidden phrases list, disclaimer

template. Write `narrator.py` Claude API client scaffold.

- Fri-Sun:** Begin Streamlit `app.py` scaffold: questionnaire page with the 7-10 questions from P3's schema (static form, no backend connection yet). Placeholder pages for Dashboard and Chat. This unblocks frontend-backend wiring in Week 3.

Dependencies this week

P3 (`rule_based.py`, callable) → P1 (FastAPI `/profile` endpoint, by Tuesday)
P2 (`OptimizationResult` interface final) → P1 (FastAPI `/optimize` endpoint)
P2 (`OptimizationResult`) → P4 (Ground Truth JSON schema uses exact field names)
P3 (SCF cluster labels) → P3's own Week 3 GBM training

Hand-off by end of Sunday 10 May:

- ☐ `/profile` and `/optimize` FastAPI endpoints returning correct JSON (P1)
- ☐ `OptimizationResult` with `risk_contributions` confirmed correct on real data (P2)
- ☐ Rule-based profiler fully tested and merged (P3)
- ☐ Ground Truth JSON schema finalised and committed (P4)
- ☐ CI still green with all new tests passing

Week 3 — 11 May to 17 May

Milestone: LLM Advisor + SCF

Goal: The conversational advisor is fully functional and validated. The 4-step post-generation validator is in place and tested. EU Awareness (Rule 9) works correctly. The backtest covers all three required historical scenarios. SCF-based GBM Phase B is completed if schedule allows.

Expected concrete outputs

Output	Owner	Required by
Validated LLM chat: 4-step Validator operational	P4	Demo, academic PDF
EU Awareness Rule 9 tested and documented	P4	Criterion 1 (problem definition), PDF section
Backtest on 3 scenarios (2008, 2020, 2022) with drawdown charts	P2	PDF section 5, Week 4 Plotly charts
<code>/advice</code> FastAPI endpoint	P1	Streamlit chat page (Week 4)

Output	Owner	Required by
GBM classifier on SCF labels with SHAP values (Phase B — if time)	P3	PDF ML section, criterion 3 advanced ML
Prompt injection defense implemented	P4	Security robustness, academic PDF
<code>test_validator.py</code> passing	P4	CI, criterion 4 (code quality)

Task breakdown by person

P1 — Backend / Data Engineering

- **Mon-Tue:** Implement `/advice` endpoint: takes `recommendation_id` from DB + user free-text message, retrieves stored `OptimizationResult`, builds Ground Truth JSON, calls `narrator.py`, runs through Validator, returns validated response. Full Pydantic typing throughout.
- **Wed-Thu:** Implement rate limiting per user on `/advice` (anti-abuse of API budget). Add prompt injection defense in `backend/llm/` (`input_sanitiser.py`): max 500 chars, keyword blocking, wrap in `<user_input>` tag. Write integration test for the full pipeline (`/profile` → `/optimize` → `/advice`).
- **Fri-Sun:** Harden DB writes: ensure every LLM call logs `validator_flags`, `retry_count`, and `fallback_triggered` to the audit table. Fix any bugs surfaced in integration tests. Help P4 with `test_validator.py` if needed.

P2 — Quant / Portfolio Optimization

- **Mon-Tue:** Implement `backend/optimizer/backtest.py`:
 - Scenario 1: GFC 2008 (Jan 2007 - Dec 2009)
 - Scenario 2: COVID crash 2020 (Jan 2020 - Dec 2020)
 - Scenario 3: Rate hike regime 2022 (Jan 2022 - Dec 2022)
 - For each: run HRP, MV, and equal-weight (1/N) strategies using the same 8 ETFs
 - Compute: cumulative return, max drawdown, Sharpe ratio, Sortino ratio
 - Include transaction cost model: 10 bps round-trip per rebalance
- **Wed-Thu:** Generate backtest result tables and drawdown series (to be plotted in Week 4). Write `test_backtest.py` with sanity checks (max drawdown ≤ 0, Sharpe finite, etc.). Export results as JSON so Plotly can consume them directly.

- **Fri-Sun:** Add regime detector scaffold (`backend/ml/regime_detector.py`): simple VIX threshold logic — if $VIX > 30 \rightarrow \text{HIGH_STRESS}$, apply ERC fallback. This triggers the "Stress Regime" UI banner in Week 4. Document regime logic in `docs/adr/ADR-003-regime-detector.md`.
-

P3 — ML / Risk Profiling

- **Mon-Tue (Phase B — priority):** Train GBM classifier (`sklearn.ensemble.GradientBoostingClassifier`) on SCF features with cluster labels from Week 2. Apply sample weights. Tune with cross-validation. Reach target validation accuracy.
- **Wed-Thu:** Compute SHAP values for the GBM model (`shap.TreeExplainer`). Extract `top_drivers` per prediction (top 3 features by absolute SHAP value). These populate the `top_drivers` field in the Ground Truth JSON, enabling the LLM narrator to explain *why* a user was classified into a given profile.
- **Fri-Sun:** Write `test_profiler.py` additions for GBM path (profile label returned, confidence score in `[0, 1]`, SHAP values sum to prediction). Update `/profile` endpoint (via P1) to use GBM when `model_version = "gbm_scf_v1"` and fall back to rule-based when GBM confidence < 0.6 .

Note: If GBM training takes longer than expected, Phase B is demoted to "future work" and the rule-based profiler remains the production path. This is acceptable — document it honestly in the PDF.

P4 — Frontend / LLM / Documentation

- **Mon-Tue:** Complete `narrator.py` and `validator.py`:
 - `narrator.py`: build prompt from Ground Truth JSON, call Claude API, return raw text
 - `validator.py`: implement all 4 validation steps:
 1. Number extraction (tolerance $\pm 2\%$ vs ground truth)
 2. Forbidden phrase scan (case-insensitive + morphological variants)
 3. Disclaimer presence check + auto-append
 4. Semantic consistency (cluster label matches cited weights)
 - Handling table: `HALLUCINATION_FLAG` $\rightarrow$ block + retry (max 2), `PRESCRIPTIVE_FLAG` $\rightarrow$ block + standard fallback, retry exhausted $\rightarrow$ generic fallback

- **Wed-Thu:** Implement and test EU Awareness Rule 9: write 5 test cases matching the patterns in `docs/ground_truth_schema.md` (EU geography question, MiFID II compliance question, USD-denominated ETF question, UCITS question, SCF limitation question). Commit `test_validator.py` with all cases passing.
- **Fri-Sun:** Wire Streamlit `app.py` questionnaire page to `/profile` endpoint. Wire the "Get Portfolio" button to `/optimize`. Display raw weights table as placeholder (charts come in Week 4). Begin drafting LaTeX PDF section 2 (ML Risk Profiler) and section 3 (Portfolio Optimization).

Dependencies this week

P1 (`/advice` endpoint) → P4 (`validator.py` integrated into `/advice`)
P2 (backtest results as JSON) → P4 (Streamlit charts, Week 4)
P3 (GBM + SHAP `top_drivers`) → P4 (Ground Truth JSON now has real `top_drivers`)
P3 (GBM confidence) → P1 (`/profile` fallback logic update)

Hand-off by end of Sunday 17 May:

- ☐ Full pipeline working end-to-end: questionnaire → profile → portfolio → LLM advice (P1 + P4)
- ☐ 4-step Validator passing `test_validator.py` (P4)
- ☐ Backtest results exported as JSON for all 3 scenarios (P2)
- ☐ GBM Phase B committed OR rule-based confirmed as final path with Phase B in future work (P3)
- ☐ CI green with all tests

Week 4 — 18 May to 24 May

Milestone: Frontend + Deploy + Hardening + Docs

Goal: The project is complete, deployed, tested, and documented. By Friday 23 May the release candidate is tagged. Saturday is reserved for final review and polish. Sunday 24 May is the hard deadline for `v1.0` release.

Expected concrete outputs

Output	Owner
Streamlit frontend: Questionnaire + Dashboard + Chat pages fully wired	P4
HRP tab + MV tab (comparison) in the Dashboard	P2, P4

Output	Owner
Plotly charts: efficient frontier, dendrogram, drawdown, weight bars	P2, P4
EU Investor Note banner on Portfolio page	P4
UCITS badges 🇪🇺 in the weights table	P4
Stress regime banner ((HIGH_STRESS) conditional)	P4
Cloud deploy (Streamlit Cloud or Railway) with CI/CD	P1
(docker-compose.yml) for local reproducibility	P1
(pytest) full suite: ≥ 80% coverage, including (test_ucits_fallback.py)	P1, P3
LaTeX PDF (5-8 pages): all sections complete	P4
(AGENTS.md) final: agent roles, PR log, evidence	P4
AI agent PR: GitHub Actions + Claude API auto-generating optimizer docstrings	P1, P4
(README.md) complete with installation, usage, API docs	P1, P4
Git tag (v1.0) + GitHub Release	P1

Task breakdown by person

P1 — Backend / Data Engineering

- **Mon-Tue:** Configure cloud deployment. Recommended: Streamlit Community Cloud (free, direct GitHub integration) + SQLite persisted volume. Alternative: Railway. Ensure all environment variables (Claude API key) are set as secrets. Update (README.md) with deployment instructions.
- **Wed-Thu:** Write (test_ucits_fallback.py) with at least 3 test cases:
 - Fallback triggers when UCITS ticker returns empty DataFrame
 - (fallback_tickers_applied) is correctly populated in (DataQualityReport)
 - DB row records the fallback correctly Write (docker-compose.yml) for local dev (SQLite volume, hot reload). Run full test suite; fix any failures. Target ≥ 80% line coverage via (pytest --cov).
- **Fri:** Set up (agent_pr.yml) GitHub Action:
 - Trigger: manual dispatch ((workflow_dispatch)) or push to (optimizer/)

- Steps: checkout repo, call Claude API with prompt "Generate NumPy-style docstrings for all public functions in `backend/optimizer/hrp.py`", commit changes to a new branch, open PR via GitHub CLI
 - Document the action in `AGENTS.md` with the PR URL as evidence
 - **Sat-Sun:** Final integration check. Tag `v1.0`. Create GitHub Release with changelog. Verify deploy is live and accessible.
-

P2 — Quant / Portfolio Optimization

- **Mon-Tue:** Produce final Plotly chart assets:
 - Efficient frontier scatter (MV vs HRP point highlighted)
 - HRP dendrogram (using `scipy.cluster.hierarchy.dendrogram`)
 - Risk contribution bar chart (one bar per ETF, colored by cluster)
 - Drawdown chart for all 3 backtest scenarios (HRP vs MV vs 1/N) Export as functions returning `plotly.graph_objects.Figure` objects for P4 to embed.
 - **Wed-Thu:** Implement MV tab in collaboration with P4: same layout as HRP tab but with Markowitz weights. Add side-by-side comparison table (HRP vs MV: Sharpe, volatility, max drawdown). This is a meaningful academic comparison that strengthens criterion 3 (scope/comprehensiveness).
 - **Fri-Sun:** Write full `docs/adr/ADR-004-ledoit-wolf-shrinkage.md` (justification for Ledoit-Wolf vs sample covariance). Final review of optimizer code: type hints complete, no magic numbers, defensive assertions in place. Assist P4 with the Portfolio Optimization section of the LaTeX PDF.
-

P3 — ML / Risk Profiling

- **Mon-Tue:** Final clean-up of `ml/profiler/` module: type hints on all public functions, NumPy docstrings, no magic numbers. Ensure `test_profiler.py` covers both rule-based and GBM (or rule-based-only if Phase B not complete) paths. Run coverage report on the `ml/` module.
- **Wed-Thu:** Write `docs/adr/ADR-005-scf-implicate-choice.md` (justification for using `implicate = 1`). Write the ML Risk Profiler section of the LaTeX PDF: describe pipeline SCF → clustering → GBM (or rule-based), explain why ML is genuine (not theatrical), include SHAP interpretation, cite Grable & Lytton 1999, Guiso et al. 2018, Federal Reserve 2022.
- **Fri-Sun:** Assist with the Backtest Results section of the LaTeX PDF (tables from P2's results). Final review of all test files. Participate in v1.0 tag and release review.

P4 — Frontend / LLM / Documentation

- **Mon-Tue:** Complete Streamlit `app.py`:
 - Page 1 — Questionnaire: 7-10 question form → POST to `/profile` → store result
 - Page 2 — Dashboard: call `/optimize`, display HRP tab (weights table with UCITS badges 📊, risk contribution bar chart, dendrogram), MV tab (comparison), EU Investor Note banner, Stress Regime banner (conditional on regime detector flag)
 - Page 3 — Chat Advisor: free-text input → POST to `/advice` → display validated LLM response with disclaimer
 - Global: MiFID II disclaimer banner above every page output
- **Wed-Thu:** Finalise LaTeX PDF:
 - Section 1: Introduction (problem, gap in existing robo-advisors, contribution)
 - Section 2: ML Risk Profiler (P3 writes, P4 integrates)
 - Section 3: Portfolio Optimization (P2 writes, P4 integrates)
 - Section 4: LLM Narrator — narrator pattern, Ground Truth JSON, 4-step Validator, EU Awareness Layer (Rule 9), prompt injection defense
 - Section 5: Backtest Results (tables and commentary)
 - Section 6: Limitations and Failure Modes — yfinance fragility, SCF US-centrism, EU/US geographic tension, HRP opacity vs Markowitz interpretability
 - Section 7: Lessons Learned — agentic workflow, AI tools used, retrospective
 - Section 8: Conclusions + GitHub link
- **Fri:** Finalise `AGENTS.md`: add AI agent PR URL and diff description as evidence. Finalise `README.md`: complete installation with `uv`, usage examples with sample output, API documentation for all three endpoints. Ensure user guide section is present (professor's requirement).
- **Sat-Sun:** End-to-end manual test of the full deployed app. Fix any UI bugs. Final PDF proofread. Submit on iCorsi.

Dependencies this week

P2 (Plotly figure functions) → P4 (embeds in Streamlit by Monday)
P1 (deploy live) → P4 (test chat page against real API)
P1 (agent_pr.yml) → P4 (AGENTS.md needs PR URL as evidence)

P3 (PDF ML section draft) → P4 (integrates into LaTeX by Wednesday)
P2 (PDF optimizer section draft) → P4 (integrates into LaTeX by Wednesday)

Final checklist by Sunday 24 May:

- ☐ Streamlit app live on cloud URL (P1)
 - ☐ `docker-compose up` works locally (P1)
 - ☐ `pytest` suite $\geq 80\%$ coverage, CI green (P1, P3)
 - ☐ `test_ucits_fallback.py` with ≥ 3 cases (P1)
 - ☐ Plotly charts: frontier, dendrogram, drawdown, risk contributions (P2)
 - ☐ HRP + MV tabs in dashboard (P2, P4)
 - ☐ EU Investor Note banner visible (P4)
 - ☐ UCITS badges  in weights table (P4)
 - ☐ Chat page: LLM response with disclaimer, validator active (P4)
 - ☐ `agent_pr.yml` ran successfully, PR visible on GitHub (P1, P4)
 - ☐ `AGENTS.md` complete with PR evidence (P4)
 - ☐ LaTeX PDF compiled to PDF, 5-8 pages, submitted on iCorsi (P4)
 - ☐ `README.md` complete with installation, usage, API docs, user guide (P1, P4)
 - ☐ Git tag `v1.0` and GitHub Release created (P1)
-

Final Deliverables Checklist

P0 — Mandatory (without these, grade ≤ 26)

- ☐ Ledoit-Wolf shrinkage on Σ
- ☐ Box constraints (0.03, 0.40) per asset, (0.10, 0.60) per cluster
- ☐ `ValidatedDataLoader` with `market_data_hash` + UCITS fallback logic
- ☐ `universe_config.py` with ETF universe UCITS-aware and fallback map
- ☐ LLM Validator 4-step with fallback handling
- ☐ Audit trail DB with all v3.1 fields
- ☐ MiFID II disclaimer above every output
- ☐ EU Investor Note on Portfolio page
- ☐ Risk contributions computed and passed to LLM
- ☐ Rule-based profiler functional (Phase A)
- ☐ `AGENTS.md` with ≥ 1 AI agent PR documented and evidenced

P1 — For an excellent grade (28-30L)

- ☐ HRP as default, MV as comparison tab
- ☐ GBM on SCF with SHAP values (Phase B)
- ☐ Regime detector with ERC fallback
- ☐ Prompt injection defense + input sanitisation

- ☐ Rule 9 (EU Awareness) implemented and tested
- ☐ Backtest on 3 historical scenarios (2008 / 2020 / 2022)
- ☐ `regulatory_context` in Ground Truth JSON with all EU fields
- ☐ Transaction cost model (10 bps) in backtest
- ☐ FastAPI with rate limiting (standard criterion element)
- ☐ UCITS badges 🇪🇺 in weights table
- ☐ `test_ucits_fallback.py` with ≥ 3 test cases

P2 — Nice to have (cite as future work)

- ☐ EWMA covariance (half-life 60 days)
- ☐ CVaR optimisation for conservative profile
- ☐ HFCS as European alternative to SCF for the profiler
- ☐ Regime-aware dynamic γ (VIX threshold)
- ☐ Counterfactual analysis per recommendation

Why the workload is balanced

Each person carries **two distinct types of responsibility** throughout the four weeks: a primary technical domain and a cross-cutting contribution to the shared codebase and documentation.

Person	Primary domain	Cross-cutting contribution	Estimated total effort
P1	Infrastructure, APIs, deploy, CI/CD	Integration testing, DB schema, AI agent PR setup	~40 h
P2	HRP optimizer, risk metrics, backtest	Plotly chart functions, ADRs for quant choices	~40 h
P3	Rule-based profiler, SCF pipeline, GBM + SHAP	ML section of LaTeX PDF, profiler test coverage	~38 h
P4	LLM narrator, Validator, Streamlit UI	LaTeX PDF assembly, AGENTS.md, README final	~42 h

P4 carries slightly more documentation burden in Week 4 (PDF assembly + README + AGENTS.md),

which is offset by lighter infrastructure work in Weeks 1-2 compared to P1.

P3's workload is front-loaded (questionnaire design + SCF preprocessing in Weeks 1-2)

but lighter in Week 4. P2 and P1 have the most technically critical deliverables

(optimizer and deploy respectively), but these are well scoped and sequenced to avoid

crunch. The plan has no single-point-of-failure week where one person blocks all others.